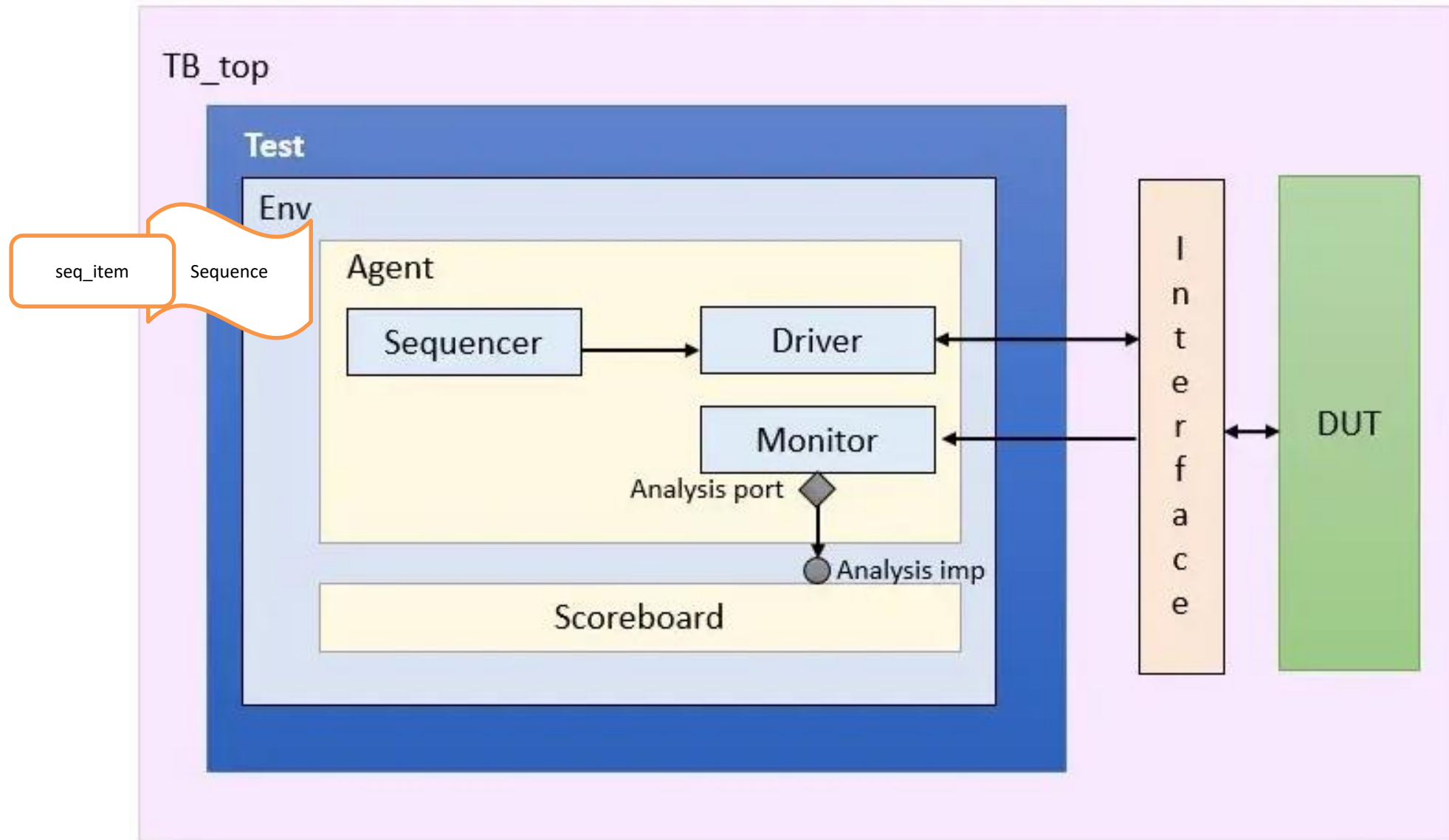


Dynamic Configuration

Yasir Farooq

Recall the UVM based Testbench

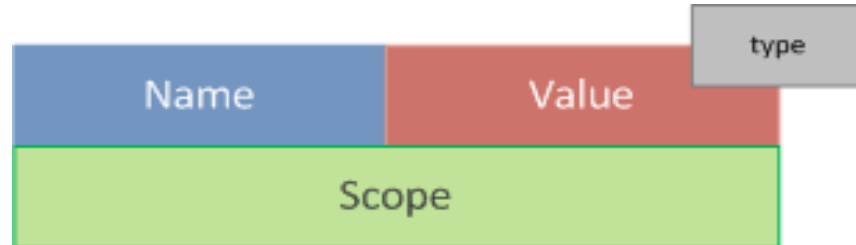


Introduction to configuration database

- UVM has an internal database which is termed as **uvm_config_db**
- It can be used to
 - Configure components
 - Supply data to sequences
 - Enable sharing of information across the disparate part of the testbench
- Configuration is stored using the scope information so its visibility can be constrained to certain parts of testbench
- Configuration database set the configuration settings for testbench components which can then be retrieved under the different names and types

Introduction to configuration database

- The `uvm_config_db` class provides a convenience on top of `uvm_resource_db` to simplify the basic interface used for UVM component instances
- Fields shown in the figure below are specified while setting and getting the configure settings for testbench components



- These are the fields used by resource database to store the information in the resource queue regarding a specific configuration
- Fields are stored in a queue by calling **set** method and can be retrieved using the **get** method

Introduction to configuration database

- For example, consider enabling functional coverage for agent in top level hierarchy

```
virtual function void build_phase (uvm_phase phase);  
.....  
uvm_config_db #(int) :: set (null , "testbench_top.env.agent" , "coverage_enable", 1);  
.....  
endfunction
```

- Agent will get configuration database settings to check if the coverage is enabled or not

```
virtual function void build_phase (uvm_phase phase);  
.....  
uvm_config_db #(int) :: get (this , "" , "coverage_enable", coverage);  
.....  
endfunction
```

Methods used with uvm_config_db

set()

- set() method is used to set configuration settings in a component for its respective hierarchical children components
- Configuration settings can be set for all or specific hierarchical children components
- cntxt, inst_name, field_name, value are fields which must be specified while setting the configuration

```
virtual function void build_phase (uvm_phase phase);  
.....  
uvm_config_db #(type) :: set (cntxt,"inst_name","field_name",value);  
.....  
endfunction
```

set()

- cntxt
 - Context is the hierarchical starting point of where the database entry is accessible
- inst_name
 - Instance name is the hierarchical path that limits accessibility of database entry
- field_name
 - Field name is the label used for the lookup of the database entry
- value
 - The value to be stored in the database of the parameterized type. By default the type is `int`

set()

Instance name is telling
that configuration is set
only for agent

Value is set to 1

```
Virtual function void build_phase ( uvm_phase phase);  
.....  
Uvm_config_db #(int) :: set (null , "testbench_top.env.agent" , "coverage_enable", 1);  
.....  
endfunction
```

Null says that the
hierarchical starting
points is uvm_root

Coverage_enable is the
field name to look for in
the instance

Type defining what type
of variable to set

get()

- get() method is used to set configuration settings in a component for its respective hierarchical children components
- cntxt, inst_name, field_name, value are fields which must be specified while getting the configuration

```
virtual function void build_phase (uvm_phase phase);  
.....  
uvm_config_db #(type) :: get (cntxt,"inst_name","field_name",value);  
.....  
endfunction
```

get()

This provides complete scope so instance name is not required here

Value is received in coverage variable

```
virtual function void build_phase (uvm_phase phase);  
.....  
uvm_config_db #(int) :: get (this , "" , "coverage_enable", coverage);  
.....  
endfunction
```

This returns complete scope of configuration setting

Type defining what type of variable to set

Coverage_enable is the field name to look for in the instance

exists()

- Checks if a value of field_name is available in inst_name, using component context as starting point
- If the field_name does not exist at the given scope name this function will return zero
- All fields in this function are same as get() and set() methods expect spell_chk field which is set to 1 to turn the spell checking on if it is expected that the field should exist in the database

```
virtual function void build_phase (uvm_phase phase);  
.....  
uvm_config_db #(virtual dut_if) :: exists (this, "*" , "dut_vif", 1);  
.....  
endfunction
```

Lab Exercise

Lab 5 Using UVM Configuration Database

To learn how to set and get values in `uvm_config_db` with different scopes.